iscte TECNOLOGIAS E ARQUITETURA

GESTÃO
ESTAGIOS

Engenharia Informática
Turno:03724PL04

129869 → Diogo Alexandre Leonardo Gonçalves
129871 → David Pedro Pereira Costa Cesário de Sousa
129833 → Martim Homem Cardoso Lopes de Almeida

Sumário

1. Introdução 2

2. Modelo de Classes de Objetos 3

2.1. Diagrama de Classes 3

2.2. Restrições 3

2.3. Notas Explicativas 4

3. Modelo Relacional 6

3.1. Esquema relacional 6

3.2. Estratégia de Integridade Referencial 7

3.3. Otimizações Possíveis 12

3.4. Dicionário de Dados 14

4. Conclusão 22

1. Introdução

Para a primeira parte do projeto da unidade curricular de "Bases de Dados", foi proposta a realização de um sistema de informação (SIEstágios) para garantir a gestão dos estágios de uma escola profissional.

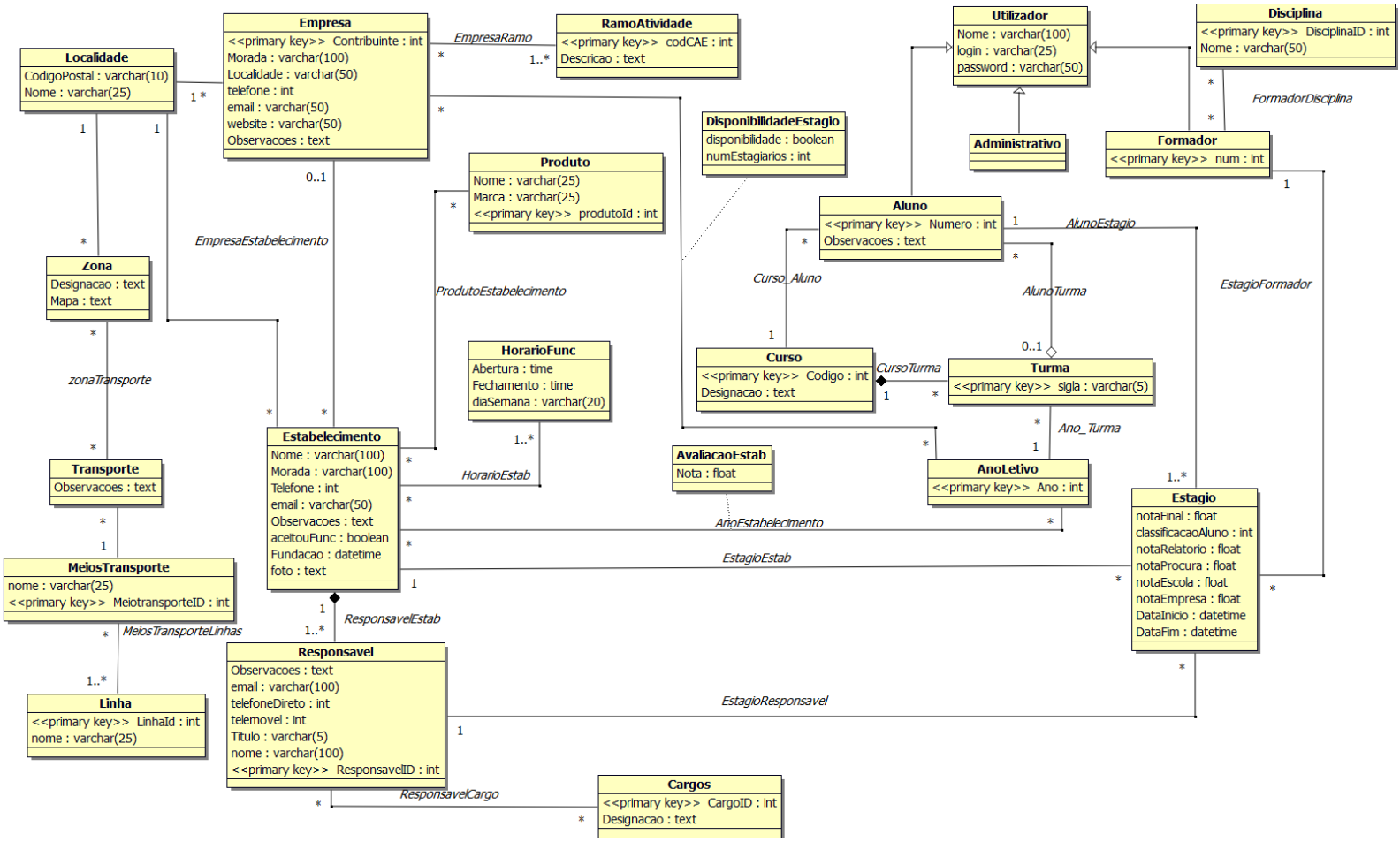
A partir da informação obtida nas diversas aulas teóricas e práticas, foi possível, em grupo, desenvolver este sistema. O trabalho consiste num modelo de classes de objetos onde para a elaboração utilizou-se o Bouml e num modelo relacional. Para a transposição do modelo UML para o relacional, utilizou-se o UMLtoRel, e o XAMPP, com o seu componente MySQL e o interface phpMyAdmin, foi utilizado para a manipulação e testes da base de dados.

O projeto começou por um debate entre os membros do grupo e depois pela elaboração do Diagrama de Classes, de forma a obter uma representação gráfica do modelo solicitado. A partir deste, com o auxílio do UMLtoRel, foi efetuada a passagem para o Modelo Relacional, onde foram realizadas alterações necessárias para garantir que o esquema final se enquadrasse totalmente com o pedido e a integridade referencial.

Este Caderno de Análise detalha a seguir estes modelos falados, começando pelas restrições e notas explicativas do Diagrama de Classes, e finalizando com a transposição para o Esquema Relacional e gestão da sua integridade.

2. Modelo de Classes de Objetos

2.1. Diagrama de Classes



2.2. Restrições

No desenvolvimento do Projeto, foram implementadas algumas restrições para garantir a integridade e a uniformidade dos dados. Sendo estas divididas em duas categorias:

Restrições feitas por meio de Classes

Algumas restrições são garantidas diretamente pelo modelo de classes, utilizando classes que limitam os valores possíveis:

- **Classe AnoLetivo** → impede que seja definido um ano letivo inválido, garantindo que apenas são aceites os valores 1, 2 ou 3.
- **Classe MeiosTransporte** → assegura que apenas meios de transporte válidos (autocarro, comboio, metro, etc.) possam ser registados, mantendo a uniformidade dos dados.

- **Classe Linha** → garante que apenas linhas existentes podem ser associadas a meios de transporte.
- **Classe Cargos** → assegura a consistência dos cargos dos responsáveis dos estabelecimentos.
- **Classe HorarioFunc** → uniformiza os horários dos estabelecimentos, garantindo que todos sigam o mesmo formato.

Restrições resolvidas em código

Outras restrições dependem de verificações durante a execução do sistema:

- O número de alunos em turmas ativas deve ser mantido entre 10 e 28.
- As notas atribuídas pelos formadores e empresas devem estar entre 0 e 20 e a nota final e a media das notas do Aluno.
- A classificação atribuída pelos alunos aos estabelecimentos deve estar entre 1 e 5.
- Cada estágio só pode existir se o aluno e o estabelecimento existirem, e o número de estagiários por estabelecimento não pode ultrapassar a capacidade máxima.
- As datas de início e fim do estágio devem ter sentido, ou seja, a data de fim tem de ser posterior à data de início.

Em suma estas restrições garantem a consistência, fiabilidade e legibilidade dos dados no sistema SIEstágios.

2.3. Notas Explicativas

O sistema SIEstágios é estruturado em várias classes, organizadas de forma a refletir as entidades e relações presentes na gestão de estágios, para tal foi usado o Bouml.

A classe **Utilizador** contém os atributos nome, login e password e funciona como superclasse para três subclasses: **Formador, Administrativo e Aluno**. A classe **Formador** possui o atributo numero e está ligada à classe **Disciplina** (disciplinaID, nome) numa relação muitos para muitos, permitindo que um formador lecione várias disciplinas e que cada disciplina possa ser lecionada por múltiplos formadores. A classe **Aluno** possui os atributos numero e observações, estando ligada a **Curso** (codigo, designacao) numa relação muitos para 1, a **Turma** (sigla) numa agregação de 0..1 para * (cada aluno pertence a uma ou nenhuma turma pois podemos ter o caso onde um aluno

recem inscrito não tenha ainda sido associado a uma turma, ou por exemplo um aluno que mude de turma pode ficar sem turma por um tempo por isso achamos melhor considerar 0..1, e uma turma tem muitos alunos) e o **Estágio**(Aluno_Numero,Formador_num,Responsavel_Estabelecimento_Estabelecimento_ID,Responsavel_ResponsavelID,Estabelecimento_Estabelecimento_ID,Estagio_ID,notaFinal,classificacaoAluno,notaRelatorio,notaProcura,notaEscola,notaEmpresa,DataInicio,DataFim) numa relação 1 para 1..*, indicando que cada aluno pode realizar vários estágios. A classe **Administrativo** herda apenas os atributos de Utilizador.

A classe **Turma** está associada a **Curso** numa relação muitos para 1, refletindo que cada turma pertence a um curso (Composição), (i.e., Turma 13 de LEI e turma 13 de LETI), e está ligada a **AnoLetivo** (ano) numa relação muitos para 1, indicando o ano do curso a que a turma corresponde. A classe **Estágio**, além de estar ligada a Aluno e Formador, está associada a Estabelecimento numa relação muitos para 1 e a **Responsável** (responsavelID, nome, titulo, cargo, telefoneDireto, telemovel, email, observacoes) também numa relação muitos para 1. A classe Responsável mantém ainda uma relação * para * com **Cargo** (cargoID, designacao).

A classe **Empresa** (contribuinte, morada, localidade, telefone, email, website, observacoes) está ligada a AnoLetivo através da classe associativa **DisponibilidadeEstagio** (disponibilidade, numEstagiarios) numa relação muitos para muitos, indicando a disponibilidade da empresa para receber estagiários em cada ano letivo. A classe **Estabelecimento** (nome, morada, telefone, email, observacoes, aceitouFunc, fundacao, foto) está ligada a **AnoLetivo** através da classe associativa **AvaliacaoEstab** (nota) numa relação muitos para muitos, representando a avaliação média de cada estabelecimento por ano letivo. Cada **Estabelecimento** também está associado a **Produto** (produtoID, nome, marca) numa relação muitos para muitos, a **HorarioFunc** (abertura, fechamento, diaSemana) numa relação muitos para 1..*, e a **Empresa** numa associação 0..1, permitindo que um **Estabelecimento** seja criado mesmo que a empresa ainda não exista.

A classe **Estabelecimento** mantém ainda uma composição com **Responsável** numa relação 1..* para 1, refletindo que cada responsável está intrinsecamente ligado ao estabelecimento. A classe **Empresa** possui ligações adicionais a **RamoAtividade** (codCAE, descricao) numa relação muitos para 1..* e a Localidade (codPostal, nome) numa relação muitos para 1. A classe **Localidade** está ligada a **Zona** (designacao, mapa) numa relação 1 para muitos, enquanto **Zona** se relaciona com **Transporte** (observacoes) numa relação muitos para muitos. A classe **Transporte** está associada a **MeioTransporte**

(meioTransporteID, nome) numa relação muitos para 1, sendo que **MeioTransporte** se liga a **Linha** (linhaID, nome) numa relação muitos para 1..* .

3. Modelo Relacional

3.1. Esquema relacional

Logical Schema (esquema relacional)

Utilizador(Utilizador_ID:int(11),Nome:varchar(100),Login:varchar(25), password:varchar(50))
Formador(num: int(11), *Utilizador_Utilizador_ID*: int(11))
Disciplina(DisciplinaID: int(11), Nome: varchar(50))
formadordisciplina(*Formador_num_*: int(11), *Disciplina_DisciplinaID_* : int(11))
administrativo(*Utilizador_Utilizador_ID*: int(11))
aluno(Numero: int(11), *Utilizador_Utilizador_ID*: int(11), *Curso_Codigo*: int(11), *Turma_Curso_Codigo*: int(11), *Turma_sigla*: varchar(5), Observacoes: text)
turma(sigla: varchar(5),*Curso_Codigo* : int(11), *AnoLetivo_Ano:* int(11))
curso(Codigo: int(11), Designacao: text)
AnoLetivo(Ano: int(11))
estagio(Estagio_ID: int(11),*Aluno_Numero*: int(11), *Formador_num*: int(11), *Estabelecimento_Estabelecimento_ID* : int(11), DataInicio: datetime, DataFim: datetime, notaEmpresa: float, notaEscola: float, notaProcura: float, notaRelatorio: float, notaFinal: float, classificacaoAluno: int(11), Responsavel_Estabelecimento_Estabelecimento_ID: int(11), Responsavel_ResponsavelID: int(11))
avaliacaoestab(*AnoLetivo_Ano_*: int(11), *Estabelecimento_Estabelecimento_ID_*: int(11), Nota: float)
Estabelecimento(Estabelecimento_ID: int(11), *Empresa_Contribuinte*: int(11), *Localidade_Localidade_ID*: int(11), Nome: varchar(100), Morada: varchar(100), Telefone : int(11), email: varchar(50), Observacoes: text, aceitouFunc: boolean, Fundacao: datetime, foto: text)
horarioestab(*Estabelecimento_Estabelecimento_ID_* : int(11), *HorarioFunc_HorarioFunc_ID_* : int(11))
horarioFunc(HorarioFunc_ID: int(11), Abertura: time, Fechamento: time, diaSemana: varchar(20))
produtoestabelecimento(*Estabelecimento_Estabelecimento_ID_* : int(11), *Produto_produtoId_* : int(11))
produto(produtoId: int(11), Nome: varchar(25), Marca: varchar(25))
Responsavel(ResponsavelID: int(11), *Estabelecimento_Estabelecimento_ID* : int(11), Observacoes: text, email: varchar(100), telefoneDireto: int(11), telemovel: int(11), Titulo: varchar(5), nome: varchar(100))

responsavelcargo(*Responsavel_Estabelecimento_Estabelecimento_ID_* : int(11), *Responsavel_ResponsavelID_* : int(11), *Cargos_CargoID_* : int(11))
cargos(CargoID: int(11), Designacao: text)
empresa(Contribuinte: int(11),*Localidade_Localidade_ID*: int(11), Morada: varchar(100), Localidade: varchar(50), telefone: int(11), email: varchar(50), website : varchar(50), Observacoes: text)
empresaramo(*Empresa_Contribuinte_*: int(11), *RamoAtividade_codCAE_* : int(11))
ramoatividade(codCAE: int(11), Descricao: text)
disponibilidadeestagio(*AnoLetivo_Ano_* : int(11), *Empresa_Contribuinte_* : int(11), disponibilidade: boolean, numEstagiarios: int(11))
Localidade(Localidade_ID: int(11), CodigoPostal: varchar(10), Nome: varchar(25))
zona(Zona_ID: int(11), *Localidade_Localidade_ID* : int(11), Designacao: text, Mapa: text)
Transporte(Transporte_ID : int(11), *MeiosTransporte_transporteID*: int(11), Observacoes: text)
zonaTransporte(Zona_Zona_ID_ : int(11), Transporte_Transporte_ID_: int(11))
meiostransporte(MeiotransporteID: int(11), nome: varchar(25))
meiostransportelinha(MeiosTransporte_MeiotransporteID_: int(11), Linha_LinhaId_ : int(11))
Linha(LinhaId: int(11), nome: varchar(25))

3.2. Estratégia de Integridade Referencial

Para a execução deste tópico do relatório, optou-se por dividi-lo em três partes. A seleção de uma estratégia eficaz de integridade referencial é importante, não apenas para este como para outros modelos de dados. Isso garante que as relações entre as várias tabelas permaneçam válidas e consistentes durante o tempo de vida do modelo, prevenindo a existência de dados órfãos ou a perda de informações relevantes.
O modelo relacional implementado pelo nosso grupo é constituído por 38 chaves estrangeiras. Estas chaves seguem, salvo raras exceções, três regras principais:

- **Modelo de Herança:**
  Para as relações de herança, foi definido CASCADE ON DELETE e CASCADE ON UPDATE, de forma a garantir que, quando a superclasse é eliminada ou atualizada, as classes que a implementam são automaticamente atualizadas, mantendo a coerência do modelo. (i.e., se apagarmos o utilizador referente ao Aluno A, não faz sentido que este continue no sistema).
- **Classes Associativas (N:M):**
  Para todas as tabelas originadas a partir de classes associativas, exceto "HorarioEstab", mantém-se a consistência com CASCADE ON DELETE e CASCADE ON UPDATE.
  Foi escolhida esta abordagem para priorizar a limpeza de dados, garantindo que, quando são removidos registos de uma das tabelas que origina a associativa, esses dados são automaticamente removidos da tabela: (i.e., Formador Carlos,

Disciplina Cálculo — existe a classe “FormadorDisciplina”. Se apagarmos o Formador, a linha [Carlos, Cálculo] perde o sentido, e por isso é removida).

- **Relações (1:N):**
  Em relações 1 para muitos, nas quais uma das classes necessita obrigatoriamente de uma referência para um objeto de outra tabela, manteve-se a regra RESTRICT ON DELETE e CASCADE ON UPDATE.
  Deste modo, não é permitido remover uma instância que esteja a ser utilizada por outra tabela, protegendo assim o modelo de dados contra a perda de informação crítica, (i.e., Turma com AnoLetivo — a turma tem de ter um ano letivo, portanto, caso existam turmas associadas a um dado ano letivo, este não poderá ser removido). No entanto esta regra é ignorada quando a relação modelada é uma Composição ou Agregação. No primeiro caso temos ON DELETE e ON UPDATE CASCADE, já no segundo fica SET NULL e CASCADE.

Relação	Chave Estrangeira	ON DELETE	ON UPDATE	Justificação
Utilizador -> Formador	Utilizador_Utilizador_ID	CASCADE	CASCADE	Sabendo que Utilizador é uma superclasse, faz sentido que, ao ser eliminado, o objeto também seja eliminado, pois este sozinho não faz sentido. Quando houver uma alteração no id de Utilizador, faz sentido que essa alteração se traduza na classe que herda.
Utilizador -> Aluno	Utilizador_Utilizador_ID	CASCADE	CASCADE	Mesmo princípio de Herança
Utilizador -> Administrativo	Utilizador_Utilizador_ID	CASCADE	CASCADE	Mesmo princípio de Herança
Turma -> Aluno	Turma_Curso_Codigo	SET NULL	CASCADE	Dado que a turma faz parte de Aluno e os alunos fazem parte de Turma, quando uma turma tem alunos associados, pode-se permitir a sua remoção, colocando o Aluno sem Turma. Já quando ocorre alguma alteração na turma, essa alteração deve refletir-se também em Aluno.
Turma -> Aluno	Turma_sigla	SET NULL	CASCADE	Ideia análoga ao explicado em cima.
Curso -> Aluno	Curso_Codigo	RESTRICT	CASCADE	Como Curso é essencial, impede-se a eliminação de um curso enquanto houver alunos ligados (ativos ou históricos), mas aceita a alteração da PK atualizando automaticamente.

Curso -> Turma	Curso_Codigo	CASCADE	CASCADE	Devido à composição, a turma precisa do curso para existir, por isso a eliminação de um curso faz sentido que apague todas as turmas desse curso, não apagando caso haja alunos no curso e na turma. Caso se altere algo do curso, de forma análoga em cima, é alterado na Turma de forma automática.
AnoLetivo -> Turma	AnoLetivo_Ano	RESTRICT	CASCADE	Dado que a turma pertence a um ano letivo, ou seja, AnoLetivo é uma referência temporal, não faz sentido remover, por exemplo, o 1.º ano letivo enquanto existirem turmas associadas a esse ano. Já a alteração é permitida.
Formador -> FormadorDisciplina	Formador_num_	CASCADE	CASCADE	Sendo FormadorDisciplina uma classe associativa, faz sentido que, se um Formador for apagado, a relação FormadorDisciplina também seja eliminada. Da mesma forma, caso haja uma alteração num Formador, é compreensível que essa mudança se reflita na relação.
Formador -> FormadorDisciplina	Disciplina_DisciplinaID_	CASCADE	CASCADE	Ideia análoga à explicada acima: caso uma Disciplina seja removida do programa, faz sentido que a relação entre essa disciplina e um Formador também seja eliminada.
Aluno -> Estagio	Aluno_Numero	RESTRICT	CASCADE	Sendo o Estágio a informação principal do modelo de dados, faz sentido que não seja permitida a eliminação de qualquer dado que possa comprometer a informação sobre os estágios ocorridos ou em curso. Qualquer modificação, no entanto, é permitida.
Formador -> Estagio	Formador_num	RESTRICT	CASCADE	Ideia análoga ao explicado em cima
Estabelecimento -> Estagio	Estabelecimento_Estabelecimento_ID	RESTRICT	CASCADE	Ideia análoga ao explicado em cima

Responsavel -> Estagio	Responsavel_Estabelecimento_Estabelecimento_ID	RESTRICT	CASCADE	Ideia análoga ao explicado em cima
Responsavel -> Estagio	Responsavel_ResponsavelID	RESTRICT	CASCADE	Ideia análoga ao explicado em cima
Estabelecimento -> Responsavel	Estabelecimento_Estabelecimento_ID	CASCADE	CASCADE	Sabendo que Responsável e Estabelecimento estão ligados por uma composição, podemos dizer que, caso um Estabelecimento seja apagado, faz sentido que o Responsável também seja eliminado, pois este depende do Estabelecimento para existir. Qualquer mudança é refletida automaticamente na relação.
Cargo -> ResponsavelCargo	Cargos_CargoID_	CASCADE	CASCADE	Como sabemos, ResponsávelCargo é uma classe associativa. Portanto, qualquer alteração — seja uma eliminação ou atualização — terá efeito direto nesta classe. Por exemplo, se removermos o cargo Ajudante, não faz sentido manter relações entre Responsável e esse cargo; por isso, essas relações devem ser apagadas.
Responsavel -> ResponsavelCargo	Responsavel_Estabelecimento_Estabelecimento_ID_	CASCADE	CASCADE	Ideia análoga ao explicado em cima
Responsavel -> ResponsavelCargo	Responsavel_ResponsavelID_	CASCADE	CASCADE	Ideia análoga ao explicado em cima
AnoLetivo -> AvaliacaoEstab	AnoLetivo_Ano_	CASCADE	CASCADE	Assumindo que o AnoLetivo só pode ser eliminado após a remoção de entidades superiores (Turmas), esta escolha garante que os registos de avaliação associados a esse ano sejam automaticamente limpos da tabela associativa.
Estabelecimento -> AvaliacaoEstab	Estabelecimento_Estabelecimento_ID_	CASCADE	CASCADE	Ideia análoga ao explicado em cima
AnoLetivo -> DisponibilidadeEstagio	AnoLetivo_Ano_	CASCADE	CASCADE	Ideia análoga ao explicado em cima
Empresa -> DisponibilidadeEstagio	Empresa_Contribuinte_	CASCADE	CASCADE	Ideia análoga ao explicado em cima
Estabelecimento -> HorarioEstab	Estabelecimento_Estabelecimento_ID_	CASCADE	CASCADE	Ideia análoga ao explicado em cima

HorarioFunc -> HorarioEstab	HorarioFunc_HorarioFunc_ID_	RESTRICT	CASCADE	O HorarioFunc é um dado de referência (ex: 8h-17h). Não deve ser eliminado se estiver a ser usado.
Estabelecimento -> ProdutoEstab	Estabelecimento_Estabelecimento_ID_	CASCADE	CASCADE	Se apagarmos um Estabelecimento, faz sentido que a relação EstabelecimentoProduto também seja removida.
Produto -> ProdutoEstab	Produto_produtoId_	RESTRICT	CASCADE	Apesar de Produto estar numa classe associativa, caso tentemos remover um produto que ainda esteja em stock num Estabelecimento, não faz sentido permitir a operação; por isso, essa remoção não é permitida.
Empresa -> Estabelecimento	Empresa_Contribuinte	SET NULL	CASCADE	Como um Estabelecimento pode ou não ter uma Empresa, podemos apagar a empresa associada desde que o estabelecimento passe a ter Empresa = NULL. Fora isso, são permitidas outras alterações.
Localidade -> Estabelecimento	Localidade_Localidade_ID	RESTRICT	CASCADE	Dado que a Localidade é essencial para o Estabelecimento, não faz sentido removermos uma localidade que ainda tenha estabelecimentos associados. No entanto, qualquer alteração na localidade é aceite.
Localidade -> Empresa	Localidade_Localidade_ID	RESTRICT	CASCADE	Ideia análoga ao explicado em cima
Empresa -> EmpresaRamo	Empresa_Contribuinte_	CASCADE	CASCADE	Sendo esta mais uma classe associativa, ao remover uma Empresa, faz sentido que a relação entre a empresa e um ou mais Ramos também seja apagada.
RamoAtividade -> EmpresaRamo	RamoAtividade_codCAE_	CASCADE	CASCADE	Caso seja apagado um Ramo de Atividade, não faz sentido manter empresas associadas a esse ramo; por isso, removemos todas as associações entre as empresas e esse ramo.

Localidade -> Zona	Localidade_Localidade_ID	RESTRICT	CASCADE	Como a Zona deve estar associada a uma Localidade, podemos afirmar com certeza que, se existirem zonas pertencentes a uma localidade, esta não pode ser apagada.
Zona -> ZonaTransporte	Zona_Zona_ID_	CASCADE	CASCADE	Caso seja apagada uma Zona, faz sentido que a relação de um Transporte com essa zona também seja eliminada.
Transporte -> ZonaTransporte	Transporte_Transporte_ID_	CASCADE	CASCADE	Ideia análoga ao explicado em cima
MeiosTransporte -> transporte	MeiosTransporte_MeiotransporteID	RESTRICT	CASCADE	Não faz sentido apagar um Meio de Transporte que esteja a ser usado no modelo de dados. Por exemplo, não é permitido remover um Autocarro se ainda houver autocarros registados no sistema.
MeiosTransporte -> MeiosTransporteLinha	MeiosTransporte_MeiotransporteID_	CASCADE	CASCADE	Ideia análoga ao explicado em cima noutras Classes Associativas
Linha -> MeiosTransporteLinha	Linha_LinhaId_	CASCADE	CASCADE	Ideia análoga ao explicado em cima noutras Classes Associativas

Em suma, a estratégia escolhida para a integridade referencial do modelo de dados imposto pelos docentes da unidade curricular reflete uma preocupação com a limpeza dos dados e a segurança da informação na base de dados.
Consideramos que o modelo se torna robusto ao acomodar associações de várias multiplicidades, bem como exceções como agregação e composição, assegurando que o sistema funcione de forma coerente e consistente com o comportamento esperado.

3.3. Otimizações Possíveis

Como foi visto e analisado em aula, ao contrário de um diagrama de classes, o modelo relacional não pretende ser apenas descritivo, mas também eficaz e eficiente. Quando fazemos a passagem do diagrama de classes para o modelo relacional, é assegurado que não há perda de informação e que este é coerente. No entanto, podem ser gerados modelos ineficientes.
No caso do SIEstagios, há cenários onde poderia ser feita uma otimização para garantir um modelo mais eficiente. Um exemplo seria o facto de termos a

superclasse Utilizador, onde existe uma generalização entre as classes Aluno, Formador e Administrativo. Como as classes que herdam Utilizador têm identidade própria, poderia ter sido eliminada a classe Utilizador e inseridos os atributos desta em cada uma das classes que a herdavam.

Utilizador(Utilizador_ID:int(11),Nome:varchar(100),Login:varchar(25), password:varchar(50))

Formador(num: int(11), Utilizador_Utilizador_ID: int(11))

administrativo(Utilizador_Utilizador_ID: int(11))

aluno(Numero: int(11), Curso_Codigo: int(11), Turma_Curso_Codigo: int(11), Turma_sigla: varchar(5), Utilizador_Utilizador_ID: int(11), Observacoes: text)

Otimização

Formador(num: int(11), Utilizador_Utilizador_ID: int(11), Utilizador_ID:int(11),Nome:varchar(100),Login:varchar(25),password:varchar(50))

administrativo(Utilizador_ID:int(11),Nome:varchar(100),Login:varchar(25), password:varchar(50))

aluno(Numero:int(11), Curso_Codigo: int(11), Turma_Curso_Codigo: int(11), Turma_sigla: varchar(5), Observacoes:text, Utilizador_ID:int(11), Nome:varchar(100), Login:varchar(25), password:varchar(50))

Neste projeto, para garantir a uniformidade dos dados, foram criadas algumas tabelas como HorarioFunc, Localidade, MeiosTransporte, etc. Apesar dessas classes tornarem o nosso modelo mais consistente, acabam por deixá-lo menos eficiente.

3.4. Dicionário de Dados

Tabela	Campo	Tipo de Dados	PK	FK	NULL	Check	Default	Coment.
Utilizador	Nome	varchar(100)	✗	✗	✗	—	—	Nome Proprio do Utilizador
Utilizador	Login	varchar(25)	✗	✗	✗	Unique	—	Nome de Utilizador no SIESTAGIOS
Utilizador	password	varchar(50)	✗	✗	✗	—	—	Password do Utilizador
Utilizador	Utilizador_ID	int(11)	✓	✗	✗	Unique | >0	—	Indentificador de Utilizadores
Formador	num	int(11)	✓	✗	✗	Unique | >0	—	Numero de Formador
Formador	Utilizador_Utilizador_ID	int(11)	✗	✓	✗	—	—	FK do argumento UtilizadorID da tabela Utilizador
Disciplina	DisciplinaID	int(11)	✓	✗	✗	Unique | >0	—	Indentificador da Disciplina
Disciplina	Nome	varchar(50)	✗	✗	✗	—	—	Nome da Disciplina
formadordisciplina	Formador_num_	int(11)	✓	✓	✗	—	—	FK do argumento num da tabela Formador
formadordisciplina	Disciplina_DisciplinaID_	int(11)	✓	✓	✗	—	—	FK do argumento DisciplinaID da tabela Disciplina
administrativo	Utilizador_Utilizador_ID	int(11)	✓	✓	✗	—	—	Indentificador da administrativo
aluno	Utilizador_Utilizador_ID	int(11)	✗	✓	✗	—	—	FK do argumento UtilizadorID da tabela Utilizador

aluno	Curso_Codigo	int(11)	✗	✓	✗	—	—	FK do argumento Codigo da tabela Curso
aluno	Turma_Curso_Codigo	int(11)	✗	✓	✗	—	—	FK do argumento Curso_Codigo da tabela turma
aluno	Turma_sigla	varchar(5)	✗	✓	✗	—	—	FK do argumento sigla da tabela turma
aluno	Numero	int(11)	✓	✗	✗	Unique | >0	—	Indentificador do Aluno
aluno	Observacoes	text	✗	✗	✓	—	NULL	Observações do Aluno
turma	Curso_Codigo	int(11)	✓	✓	✗	—	—	FK do argumento Codigo da tabela Curso
turma	AnoLetivo_Ano	int(11)	✗	✓	✗	—	—	FK do argumento Ano da tabela AnoLetivo
turma	sigla	varchar(5)	✓	✗	✗	Unique	—	Indentificador da Turma
curso	Codigo	int(11)	✓	✗	✗	Unique | > 0	—	Indentificador do Curso
curso	Designacao	text	✗	✗	✓	—	NULL	Designação do Curso
AnoLetivo	Ano	int(11)	✓	✗	✗	Valores entre {1,2,3}	—	Identificador do Ano Letivo
estagio	Aluno_Numero	int(11)	✗	✓	✗	—	—	FK do argumento Numero da tabela Aluno
estagio	Formador_num	int(11)	✗	✓	✗	—	—	FK do argumento num da tabela Formador
estagio	Estabelecimento_Estabelecimento_ID	int(11)	✗	✓	✗	—	—	FK do argumento Estabelecimento_ID da tabela Estabelecimento

estagio	Estagio_ID	int(11)	✓	✗	✗	Unique | > 0	—	Identificador do Estagio
estagio	DataInicio	datetime	✗	✗	✓	—	NULL	Data inicio do Estagio
estagio	DataFim	datetime	✗	✗	✓	DataFim >DataInicio	NULL	Data final do Estagio
estagio	notaEmpresa	float	✗	✗	✓	valor no intervalo [0,20]	0.0	Nota dada pela empresa
estagio	notaEscola	float	✗	✗	✓	valor no intervalo [0,20]	0.0	Noda dada pela Escola
estagio	notaProcura	float	✗	✗	✓	valor no intervalo [0,20]	0.0	Nota da procura de estágio
estagio	notaRelatorio	float	✗	✗	✓	valor no intervalo [0,20]	0.0	Nota do Relatorio
estagio	notaFinal	float	✗	✗	✓	valor no intervalo [0,20]	0.0	Nota Final do Estagio
estagio	classificacao Aluno	int(11)	✗	✗	✓	valor no intervalo [1,5]	1.0	Classificação do local onde houve o estagio
estagio	Responsavel _Estabelecim ento_Estabel ecimento_ID	int(11)	✗	✓	✗	—	—	FK do argumento Estabelecim ento_Estabel ecimento_ID da tabela Responsavel
estagio	Responsavel _Responsave lID	int(11)	✗	✓	✗	—	—	FK do argumento ResponsavelI D da tabela Responsavel
avaliacaoest ab	AnoLetivo_An o_	int(11)	✓	✓	✗	—	—	FK do argumento Ano da tabela AnoLetivo
avaliacaoest ab	Estabelecim ento_Estabel ecimento_ID _	int(11)	✓	✓	✗	—	—	FK do argumento Estabelecim ento_ID da tabela Estabelecim ento

avaliacaoestab	Nota	float	✗	✗	✗	valor no intervalo [1,5]	0.0	Nota media dada pelos varios alunos num dado ano letivo
Estabelecimento	Empresa_Contribuinte	int(11)	✗	✓	✗	—	—	FK do argumento Contribuinte da tabela Empresa
Estabelecimento	Localidade_Localidade_ID	int(11)	✗	✓	✗	—	—	FK do argumento Localidade_ID da tabela Localidade
Estabelecimento	Estabelecimento_ID	int(11)	✓	✗	✗	Unique | > 0	—	Identificador do Estabelecimento
Estabelecimento	Nome	varchar(100)	✗	✗	✗	—	—	Nome do Estabelecimento
Estabelecimento	Morada	varchar(100)	✗	✗	✗	Unique	—	Morada do Estabelecimento
Estabelecimento	Telefone	int(11)	✗	✗	✗	Unique	—	Telefone do Estabelecimento
Estabelecimento	email	varchar(50)	✗	✗	✗	Unique	—	email do Estabelecimento
Estabelecimento	Observacoes	text	✗	✗	✓	—	NULL	Observacoes do Estabelecimento
Estabelecimento	aceitouFunc	boolean	✗	✗	✗	—	0	aceitouFunc do Estabelecimento
Estabelecimento	Fundacao	datetime	✗	✗	✗	—	—	Fundacao do Estabelecimento
Estabelecimento	foto	text	✗	✗	✓	Caminho de Ficheiro Valido	NULL	foto do Estabelecimento

horarioestab	Estabelecimento_Estabelecimento_ID_	int(11)	✓	✓	✗	—	—	FK do argumento Estabelecimento_ID da tabela Estabelecimento
horarioestab	HorarioFunc_HorarioFunc_ID_	int(11)	✓	✓	✗	—	—	FK do argumento HorarioFunc_ID da tabela HorarioFunc
horarioFunc	HorarioFunc_ID	int(11)	✓	✗	✗	Unique | > 0	—	Identificador do Horarario
horarioFunc	Abertura	time	✗	✗	✗	—	—	Hora de Abertura
horarioFunc	Fechamento	time	✗	✗	✗	—	—	Hora de Fecho
horarioFunc	diaSemana	varchar(20)	✗	✗	✗	[seg,ter,qua,qui,sex,sab,dom]	—	Dia da Semana daquele horario
produtoestabelecimento	Estabelecimento_Estabelecimento_ID_	int(11)	✓	✓	✗	—	—	FK do argumento Estabelecimento_ID da tabela Estabelecimento
produtoestabelecimento	Produto_produtoId_	int(11)	✓	✓	✗	—	—	FK do argumento produtoId da tabela Produto
produto	Nome	varchar(25)	✗	✗	✗	—	—	Nome do Produto
produto	Marca	varchar(25)	✗	✗	✗	—	—	Marca do Produto
produto	produtoId	int(11)	✓	✗	✗	Unique | > 0	—	Identificador do Produto
Responsavel	ResponsavelID	int(11)	✓	✗	✗	Unique | > 0	—	Identificador do Responsavel
Responsavel	Estabelecimento_Estabelecimento_ID	int(11)	✓	✓	✗	—	—	FK do argumento Estabelecimento_ID da tabela Estabelecimento

Responsavel	Observacoes	text	✗	✗	✓	—	NULL	Observações do Responsavel
Responsavel	email	varchar(100)	✗	✗	✗	—	—	Email do Responsavel
Responsavel	telefoneDiretO	int(11)	✗	✗	✗	—	—	TelefoneDiretO do Responsavel
Responsavel	telemovel	int(11)	✗	✗	✗	—	—	Telemovel do Responsavel
Responsavel	Titulo	varchar(5)	✗	✗	✗	—	—	Titulo do Responsavel
Responsavel	nome	varchar(100)	✗	✗	✗	—	—	Nome do Responsavel
responsavelcargo	Responsavel_Estabelecimento_Estabelecimento_ID_	int(11)	✓	✓	✗	—	—	FK do argumento Estabelecimento_Estabelecimento_ID da tabela Responsavel
responsavelcargo	Responsavel_ResponsavelID_	int(11)	✓	✓	✗	—	—	FK do argumento ResponsavelID da tabela Responsavel
responsavelcargo	Cargos_CargoID_	int(11)	✓	✓	✗	—	—	FK do argumento CargoID da tabela Cargos
cargos	CargoID	int(11)	✓	✗	✗	Unique | > 0	—	Identificador do Cargo
cargos	Designacao	text	✗	✗	✓	—	NULL	Designação do Cargo
empresa	Localidade_Localidade_ID	int(11)	✗	✓	✗	Unique | > 0	—	FK do argumento Localidade_ID da tabela Localidade
empresa	Contribuinte	int(11)	✓	✗	✗	—	—	Contribuinte da Empresa
empresa	Morada	varchar(100)	✗	✗	✗	Unique	—	Morada da Empresa
empresa	Localidade	varchar(50)	✗	✗	✗	—	—	Localidade da Empresa
empresa	telefone	int(11)	✗	✗	✗	Unique	—	telefone da Empresa
empresa	email	varchar(50)	✗	✗	✗	Unique	—	email da Empresa

empresa	website	varchar(50)	✗	✗	✗	Unique	—	website da Empresa
empresa	Observacoes	text	✗	✗	✓	—	NULL	Observacoes da Empresa
empresaramo	Empresa_Contribuinte_	int(11)	✓	✓	✗	—	—	FK do argumento Contribuinte da tabela Empresa
empresaramo	RamoAtividade_codCAE_	int(11)	✓	✓	✗	—	—	FK do argumento codCAE da tabela RamoAtividade
ramoatividade	codCAE	int(11)	✓	✗	✗	Unique | > 0	—	Codigo CAE do Ramo de Atividade
ramoatividade	Descricao	text	✗	✗	✓	—	NULL	Descrição do ramo de Atividade
disponibilidadeestagio	AnoLetivo_Ano_	int(11)	✓	✓	✗	—	—	FK do argumento Ano da tabela AnoLetivo
disponibilidadeestagio	Empresa_Contribuinte_	int(11)	✓	✓	✗	—	—	FK do argumento Contribuinte da tabela Empresa
disponibilidadeestagio	disponibilidade	boolean	✗	✗	✗	—	0	Disponibilidade de Estagio
disponibilidadeestagio	numEstagiarios	int(11)	✗	✗	✗	—	0	Numero de Estagiarios
Localidade	Localidade_ID	int(11)	✓	✗	✗	Unique | > 0	—	Identificador de Localidade
Localidade	CodigoPostal	varchar(10)	✗	✗	✗	Unique	—	Codigo Postal da Localidade
Localidade	Nome	varchar(25)	✗	✗	✗	—	—	Nome da Localidade
zona	Localidade_Localidade_ID	int(11)	✗	✓	✗	—	—	FK do argumento Localidade_ID da tabela Localidade
zona	Zona_ID	int(11)	✓	✗	✗	Unique | > 0	—	Identificador de Zona

zona	Designacao	text	✗	✗	✓	—	NULL	Designação de zona
zona	Mapa	text	✗	✗	✗	Caminho de Ficheiro Valido	—	Mapa da Zona
Transporte	MeiosTransporte_transporteID	int(11)	✗	✓	✗	—	—	FK do argumento transporteID da tabela MeiosTransporte
Transporte	Transporte_ID	int(11)	✓	✗	✗	Unique | > 0	—	Identificador do Transporte
Transporte	Observacoes	text	✗	✗	✓	—	NULL	Observações do Transporte
zonaTransporte	Zona_Zona_ID_	int(11)	✓	✓	✗	—	—	FK do argumento Zona_ID da tabela Zona
zonaTransporte	Transporte_Transporte_ID_	int(11)	✓	✓	✗	—	—	FK do argumento Transporte_ID da tabela Transporte
meiostransporte	nome	varchar(25)	✗	✗	✗	—	—	nome do meio de transporte
meiostransporte	MeiotransporteID	int(11)	✓	✗	✗	Unique | > 0	—	Identificador do Transporte
meiostransportelinha	MeiosTransporte_MeiotransporteID_	int(11)	✓	✓	✗	—	—	FK do argumento MeiotransporteID da tabela MeiosTransporte
meiostransportelinha	Linha_LinhaId_	int(11)	✓	✓	✗	—	—	FK do argumento LinhaID da tabela Linha
Linha	LinhaId	int(11)	✓	✗	✗	Unique | > 0	—	Identificador da Linha
Linha	nome	varchar(25)	✗	✗	✗	—	—	nome da linha

4. Conclusão

Em suma, com este trabalho foi possível pôr em prática o aprendido em aula, ou seja, fazer um diagrama de classes usando o software Bouml a partir de um enunciado, como feito nas aulas práticas. Neste projeto também foi utilizado o UmlToRel para fazer a transposição do diagrama de classes para o modelo relacional. Já no phpMyAdmin foram feitas pequenas alterações consideradas necessárias para que o nosso modelo de dados ficasse o mais próximo possível do modelo esperado. Por fim, usando softwares como o Word e o Excel, construímos o caderno de análise que explica a construção tanto do diagrama de classes como do modelo relacional. Com isto, consideramos que este trabalho foi bastante importante para o desenvolvimento de conhecimentos para, a partir de um enunciado, construir uma base de dados capaz de responder ao esperado.